Asadito RG: Java original y sesiones

Guía para comprender y defender el proyecto. Actualización: 10/11/2026.

Se tomó como base el Java original del commit 3ba7c00 y como referencia el documento ModificaClient.pdf del profesor. Se conserva la interfaz actual y los paneles, con los agregados de Java necesarios para guardar información y controlar permisos. No se volvió todo el backend literalmente al original: eso habría eliminado las funciones de gestión que elegiste conservar.

1. Qué se restauró y qué se simplificó

Los DTO vuelven a sus nombres, campos, constructores y relaciones originales: LocalesDTO, ProductosDTO, PreciosDTO y ProductoPrecioDTO. No se introducen DTO de entrada para los formularios.

LocalesDTO, ProductosDTO y ProductoPrecioDTO son idénticos a la versión original. PreciosDTO tiene una única corrección en su constructor:

```
this.precio = precio.getPrecio();
```

Sin esa asignación, el DTO devolvía el valor predeterminado de float: 0. El nombre del atributo, su tipo float y el getter original se conservan.

LocalesRepository, ProductosRepository y PreciosService también quedan idénticos al original. Se recuperan los tres controladores originales y su forma de trabajar con @Autowired, repositorios, streams y DTO. Se conservan las relaciones entre Locales, Productos y Precios, y sus constructores originales.

Se retiran del backend CatalogService, OperadoresService, PresupuestoService y PdfService; los controladores separados CatalogController, OperadoresController, AuthController y PresupuestoController; y los DTO adicionales CatalogRequests, CrearOperadorRequest, Presupuesto y PresupuestoRequest. Las configuraciones anteriores SecurityConfig y AdminBootstrap se reemplazan por las clases de sesiones descriptas abajo. Ya no se utiliza PDFBox ni Bean Validation: las verificaciones pequeñas se hacen con condiciones en los métodos existentes.

El proyecto tiene 15 archivos Java de producción originales y 6 adicionales para seguridad y respuestas de error. Los tests quedan en src/test y no agregan clases al funcionamiento de la aplicación.

2. Inventario completo de archivos Java

Todas las rutas de esta sección parten de src/main/java/com/appasadito/AsaditoRg/.

Archivos originales sin cambios

Archivo	Responsabilidad
dtos/LocalesDTO.java	Respuesta original de locales, incluidas sus coordenadas y colección de precios.
dtos/ProductosDTO.java	Respuesta original de cortes.
dtos/ProductoPrecioDTO.java	Agrupación original de precios por comercio.
repositorios/LocalesRepository.java	Acceso JPA original a locales.
repositorios/ProductosRepository.java	Acceso JPA original a productos.
services/PreciosService.java	Transformación original de la consulta de últimos precios.

Archivos originales con cambios pequeños

Archivo	Cambio y motivo
dtos/PreciosDTO.java	Una asignación para devolver el precio real. No cambia su estructura.
modelos/Locales.java	Campo activo, valor inicial true y getter/setter para la baja lógica.
modelos/Productos.java	El mismo campo activo para ocultar cortes dados de baja.
modelos/Precios.java	Campo activo conservado; precio vuelve a float. Se mantiene la columna SQL numeric(14,2) mediante @Column para respetar la base de la versión previa.
controllers/LocalesController.java	GET original con filtro activo; POST, PUT y DELETE de gestión, búsqueda por ID y validación breve.
controllers/ProductosController.java	GET original con filtro activo; POST, PUT y DELETE, búsqueda por ID y validación del nombre.
controllers/PreciosController.java	GET originales; filtro de registros activos y POST de precio. Verifica importe, fecha y relaciones existentes.
repositorios/PreciosRepository.java	Misma consulta original con condiciones activo para precio, corte y local.
AsaditoRgApplication.java	Conserva los cortes y locales iniciales. Inyecta PasswordEncoder y agrega un método para crear las cuentas iniciales desde variables de entorno.

Archivos adicionales respecto del original

Archivo	Qué debés explicar
modelos/Usuario.java	Entidad con id, email, password y rol. Su password contiene el hash. Se mapea a la tabla usuarios y a la columna existente password_hash. @JsonIgnore evita incluir la contraseña en JSON.
repositorios/UsuarioRepository.java	JpaRepository, findByEmail que retorna Usuario o null, y búsqueda de operadores por rol. Spring implementa las consultas.
configurations/WebAuthentication.java	Busca por email, construye el User de Spring y ofrece el PasswordEncoder.
configurations/WebAuthorization.java	Define páginas públicas, permisos, formulario de login, cierre de sesión y respuestas HTTP.
controllers/UsuarioController.java	Registro USER, consulta de sesión, token CSRF y listado/creación de operadores por ADMIN. Comparte un método privado crear para validar y guardar el hash.
controllers/ApiErrors.java	Dos manejadores breves: ResponseStatusException conserva su estado/mensaje y un conflicto de integridad devuelve HTTP 409 sin exponer SQL.

ApiErrors y Usuario ya existían en las mejoras previas, pero no estaban en el Java original. Ahora están reducidos. No hay una capa adicional de servicios ni una jerarquía de solicitudes para estas funciones.

3. Cómo se adapta el PDF del profesor

El ejemplo del profesor usa Client y HomeBankingApplication. En este proyecto sus equivalentes son Usuario y AsaditoRgApplication. No se modifica Locales para convertir un comercio en una cuenta de acceso: representan cosas distintas.

Paso del PDF	Equivalente en Asadito RG
Agregar password a Client	Usuario posee email, password y rol.
Agregar findByEmail	UsuarioRepository declara Usuario findByEmail(String email).
Agregar spring-boot-starter-security	Dependencia en build.gradle.
Buscar el usuario y devolver User	WebAuthentication usa el repositorio y AuthorityUtils.createAuthorityList(usuario.getRol()).
Crear PasswordEncoder	PasswordEncoderFactories.createDelegatingPasswordEncoder().
Codificar contraseñas iniciales	AsaditoRgApplication usa passwordEncoder.encode antes de guardar.
Configurar rutas y formulario	WebAuthorization define /api/login y /api/logout; parámetros email y password.
Responder al frontend sin redirección	Login correcto y logout devuelven 204; login incorrecto devuelve 401.

Se usa SecurityFilterChain y un bean UserDetailsService, siguiendo la alternativa actualizada que aparece al final del material. No se necesitan las clases antiguas WebSecurityConfigurerAdapter ni GlobalAuthenticationConfigurerAdapter. La búsqueda por email y la creación del User cumplen la misma función del ejemplo.

Se utilizan authorities ADMIN, OPERADOR y USER, sin prefijo ROLE_. Por eso las reglas usan hasAuthority("ADMIN") y no hasRole("ADMIN"). No mezcles ambas convenciones al modificar permisos.

Hay dos diferencias deliberadas respecto de algunos ejemplos del material: se conserva CSRF porque se usan cookies de sesión, y no se desactiva la protección de frames porque la aplicación no necesita embeberse en otras páginas.

4. Recorrido del inicio de sesión

1. El navegador pide GET /api/auth/csrf. Spring entrega un token y el nombre de su cabecera. Esto puede crear una sesión anónima.

2. session.js envía POST /api/login con email y password como formulario URL-encoded, más la cabecera CSRF.

3. El filtro de Spring busca el usuario mediante WebAuthentication y UsuarioRepository.findByEmail.

4. PasswordEncoder compara la contraseña ingresada con el hash guardado. Tu controlador no compara contraseñas ni genera tokens de autenticación.

5. Si coincide, Spring autentica al usuario, cambia el ID de sesión para evitar fijación de sesión y responde 204. El navegador conserva la cookie JSESSIONID.

6. GET /api/auth/me devuelve authenticated, email, admin y operador para dibujar la interfaz. El frontend renueva el token CSRF después del ingreso.

7. En cada operación, Spring comprueba la sesión y las reglas de WebAuthorization. Ocultar un botón no otorga ni revoca permisos.

8. POST /api/logout invalida la sesión y elimina la cookie. La sesión también vence tras 30 minutos de inactividad.

La contraseña nunca se almacena en localStorage. La cookie es HttpOnly y SameSite=Lax. Para un despliegue HTTPS se configura SESSION_COOKIE_SECURE=true.

El PasswordEncoder nuevo genera hashes con prefijo {bcrypt}, como en el PDF. Se agregaron dos líneas de compatibilidad con los hashes BCrypt anteriores sin prefijo para que las cuentas existentes sigan entrando. No se aceptan contraseñas almacenadas en texto plano y no se cambia la contraseña de cuentas existentes.

5. Permisos y endpoints

Función	Método y ruta	Permiso
Catálogo	GET /api/productos, /api/locales, /api/precios y /api/precios/ultimos	Público
Crear cuenta común	POST /api/auth/registro	Público; el servidor asigna USER
Inicio / cierre	POST /api/login y /api/logout	Gestionados por Spring; requieren CSRF
Estado y CSRF	GET /api/auth/me y /api/auth/csrf	Público; solo informa la sesión actual
Agregar corte/local	POST /api/admin/productos o /api/admin/locales	ADMIN
Modificar corte/local	PUT /api/admin/productos/{id} o /api/admin/locales/{id}	ADMIN
Baja de corte/local	DELETE en las mismas rutas con {id}	ADMIN
Publicar precio	POST /api/admin/precios	OPERADOR
Listar / crear operadores	GET / POST /api/admin/operadores	ADMIN

Nadie edita ni elimina precios publicados. ADMIN consulta precios, pero no los carga. Las demás rutas /api/admin/** se deniegan. Todas las escrituras, incluido registro, login y logout, llevan CSRF.

Los formularios usan @RequestParam como en el ejemplo del profesor. session.js convierte los campos planos de los formularios a URLSearchParams. No se agregan clases Java para representar cada formulario. Un cliente externo debe enviar application/x-www-form-urlencoded, no el JSON de la implementación anterior.

Un 400 significa datos inválidos; 401, falta de sesión o credenciales incorrectas; 403, falta de permiso o token CSRF inválido; 404, registro inexistente/inactivo; 409, email repetido o conflicto con la base. El frontend muestra el resultado mediante mensajes flotantes.

6. Cómo se guardan los cambios del panel

Cortes y locales

El controlador recibe los parámetros, hace comprobaciones cortas, busca por ID cuando corresponde y llama save del repositorio. Devuelve el DTO original. Los nombres se recortan con trim; se rechazan campos vacíos y textos demasiado largos. Para locales se verifican latitud y longitud finitas, con rangos de -90 a 90 y -180 a 180. Se guardan en coordenada_x y coordenada_y, como en el modelo original.

El método eliminar no llama delete: busca el objeto, ejecuta setActivo(false) y guarda. Por eso se conservan sus relaciones e historial. Los GET directos filtran los registros activos. Las colecciones incluidas en los DTO se conservan como estaban originalmente; no se diseñó una nueva respuesta para cada pantalla.

Precios

El operador envía localId, productoId, precio y fecha. PreciosController verifica precio positivo y finito, formato de fecha, que no sea futura y que existan el corte y el local activos. Crea un Precios usando el constructor original y guarda las relaciones. Publicar un nuevo valor conserva los anteriores.

La calculadora del frontend toma el más reciente por fecha y, ante empate, el ID más alto. El endpoint /api/precios/ultimos y su servicio original se conservan por compatibilidad: su consulta puede devolver varias filas si la fecha empata y agrupa por nombre de comercio. La interfaz actual usa /api/precios y resuelve la selección por IDs en JavaScript para no modificar ese servicio.

Se mantiene float en Precios y PreciosDTO por tu pedido de conservar el Java original. float tiene precisión limitada: a importes elevados puede perder centavos. La columna SQL numeric(14,2) conserva el esquema anterior, pero no elimina ese límite del tipo Java. Para una futura versión contable convendría BigDecimal; no se vuelve a introducir ahora sin que lo decidas.

Operadores y registro

UsuarioController normaliza el email, valida email y contraseña, verifica duplicados, codifica el hash y guarda Usuario. El registro público fija USER y el endpoint administrativo fija OPERADOR: un parámetro rol enviado desde afuera no altera esa elección. El listado de operadores solo devuelve email y rol. La confirmación de contraseña se comprueba en el formulario y no necesita otro campo Java.

7. Qué pasó al frontend

Archivo JavaScript	Responsabilidad
session.js	Login, logout, estado de sesión, CSRF, solicitudes de formularios y mensajes de error.
gestion.js	Apartados del panel, filtros, formularios y mensajes flotantes; conserva la interfaz que ya utilizabas.
presupuesto.js	Adapta los DTO originales, elige precios recientes y calcula cantidades en incrementos de 0,5 kg.
app.js	Comparador, selector de corte en móvil, ranking por total, selección del presupuesto y WhatsApp.
reporte.js	Genera el PDF con logo, tipografía, detalle, total y paginación mediante jsPDF.

La selección y los presupuestos se calculan en el navegador. Se redondea a centavos y se suma con enteros para evitar errores acumulativos del cálculo de presentación. Cada local debe tener precio de todos los cortes seleccionados para participar de la comparación. No se reemplaza un precio faltante por cero.

Ya no existen endpoints Java /api/presupuestos/* ni un servicio Java para generar PDF. jsPDF, las fuentes y el logo se sirven desde /assets, sin CDN. El botón de descarga consulta la sesión para conservar el flujo de ingreso de la interfaz. Al ser un archivo armado en el navegador, es un presupuesto orientativo que puede modificarse por quien controla el navegador: no es un documento certificado ni una factura protegida por el servidor.

Los permisos, el hash de contraseñas, la validación de datos guardados y las escrituras siguen en Java. Esas responsabilidades no pueden trasladarse a JavaScript como mecanismo de seguridad.

WhatsApp abre wa.me con un resumen de texto. Cuando el dispositivo lo permite se usa Web Share para compartir el archivo PDF; en los demás casos se descarga y se adjunta manualmente. No se necesita Java adicional ni se envían mensajes automáticamente.

8. Configurarlo en IntelliJ

1. Abrí Run → Edit Configurations y seleccioná la configuración que realmente ejecutás.

2. En Environment variables cargá SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME y SPRING_DATASOURCE_PASSWORD para tu base.

3. Solo para crear el primer administrador, agregá ASADITO_ADMIN_EMAIL y ASADITO_ADMIN_PASSWORD. El email debe ser válido y la contraseña tener al menos 10 caracteres y hasta 72 bytes UTF-8.

4. En el editor por filas de IntelliJ ingresá los valores sin comillas. La contraseña 123456 no cumple el mínimo.

5. Detené la ejecución anterior, reiniciá e ingresá en /gestion.html. Luego retirá las dos variables de creación inicial y reiniciá: la cuenta ya queda guardada.

6. Como ADMIN, abrí el apartado Operadores y creá las cuentas que necesites. No requiere modificar Java ni agregar variables por cada operador.

También se conservan ASADITO_OPERADOR_EMAIL y ASADITO_OPERADOR_PASSWORD como alternativa opcional para una cuenta inicial. El email debe ser distinto del administrador.

Las variables de creación inicial NO restablecen claves existentes. Si un email ya está guardado, su hash sigue siendo el mismo. Si pertenece a otro rol, se informa el conflicto. No se vuelve a crear ni se convierte una cuenta automáticamente. Los usuarios existentes conservan su tabla, rol e identificadores.

No hay que borrar tablas para aplicar esta simplificación. Si tu base ya funcionaba con la versión anterior, conserva activo y password_hash. Si venís de una base anterior a esas mejoras, revisá el README y la migración antes de ejecutarla; no es un paso que debas repetir en una base actualizada.

9. Ejercicio para explicarlo en clase

Primero seguí una lectura pública: GET /api/productos → ProductosController → ProductosRepository → ProductosDTO → JSON → app.js. Mostrá que el DTO original controla qué propiedades se responden.

Después explicá el login con estas preguntas: ¿qué busca findByEmail?, ¿qué contiene password?, ¿quién compara la clave?, ¿qué conserva la cookie?, ¿qué diferencia hay entre autenticación y autorización? La autenticación determina quién ingresó; la autorización determina qué acciones puede hacer.

Como ADMIN, creá y modificá un corte y un local. Dales de baja y comprobá en la base que activo pasa a false. Creá un operador, cerrá sesión e ingresá con esa cuenta: solo debe poder publicar precios.

Publicá dos precios del mismo corte/local. Mostrá que ambos registros quedan guardados y el comparador elige el último. Elegí 1,5 kg y explicá multiplicación, comparación entre comercios y generación del PDF en JavaScript.

Intentá enviar una escritura sin CSRF o publicar un precio con un ADMIN: debe rechazarse aunque manipules los botones. Así demostrás que el control de acceso no depende de la interfaz.

Para practicar una modificación sencilla, cambiá el máximo de caracteres de nombre en validarNombre y en el formulario. Antes identificá ambos lugares y luego probá un nombre que supere el límite.

10. Verificación realizada y límites

Se ejecutó Gradle build con 11 pruebas de Java satisfactorias: arranque, DTO, login con hashes nuevos y anteriores, sesión/logout, CSRF, permisos, paneles, validaciones y creación de cuentas. Usan H2 aislada en modo PostgreSQL.

También se probó en PostgreSQL y Chromium con una base temporal: ADMIN crea un operador, el operador ingresa y publica un precio, la calculadora obtiene el total para 1,5 kg, se descarga un PDF sin llamar endpoints de presupuestos y el móvil presenta tarjetas. Se verificaron cálculos de centavos, incrementos de 0,5 kg y selección por fecha/ID con pruebas de JavaScript.

Se conserva Spring Boot 2.7 y javax para respetar la base del curso. Recuperación de contraseñas, cambios de rol, edición de operadores, mensajería automática y optimización para catálogos grandes no se agregaron. Actualmente se cargan los precios para procesarlos en el navegador; si el historial crece mucho se puede estudiar paginación sin cambiar los DTO de esta entrega.

Esta guía documenta los cambios; para defenderlos conviene ejecutar los ejercicios y recorrer los métodos en IntelliJ. El material del profesor orienta la implementación, y los criterios de evaluación siguen dependiendo de la cátedra.